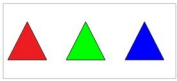

Figure 2: Three coloured triangles drawn with PGF/TikZ

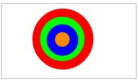

Figure 3: Four coloured concentric circles drawn with PGF/TikZ

command. For example, there are options to change the colour of the line from the default black; to vary the thickness of the line being drawn, etc. Replace the three lines of code inside the *tikzpicture* environment of the script *pgf2.tex* with the lines of code *\draw[yellow, very thick, fill=red] (0,1)--(2,1)--(1,3)--cycle;*, *\draw[yellow, very thick, fill=green] (3,1)--(5,1)--(4,3)--cycle;* and *\draw[yellow, very thick, fill=blue] (6,1)--(8,1)--(7,3)--cycle*; to obtain *pgf3.tex*. On executing the command, *pdflatex pgf3.tex*, this script will produce an output document named *pgf3.pdf*, which has three triangles drawn with thick yellow coloured lines and filled with red, green and blue colour. The script *pgf3.tex* is also available for download.

Drawing circles with PGF/TikZ

Now let us look at how to draw circles using PGF/TikZ. Consider the script *pgf4.tex* shown below.

```
\documentclass{article}
\usepackage{tikz}
\begin{document}
\begin{tikzpicture}
     \draw[fill=red]       (0,0) circle (4cm);
     \draw[fill=green]   (0,0) circle (3cm);
     \draw[fill=blue]     (0,0) circle (2cm);
     \draw[fill=orange] (0,0) circle (1cm);
\end{tikzpicture}
\end{document}
```

The only line of code that requires an explanation is *\draw[fill=red] (0,0) circle (4cm);*. This line draws a red circle of radius 4cm with its centre at point (0, 0) and fills this circle with red colour. The remaining three lines in the *tikzpicture* environment of LaTeX draw three more circles with a radius of 1cm less than the previous one and with the centre at the same point, so that we have four concentric circles drawn in the final output file. On executing the command *pdflatex pgf4.tex* in a terminal, four concentric circles are drawn in red, green, blue and orange colours. Figure 3 shows the output of the script *pgf4.tex*.

Remember that components of an image with multiple layers are drawn in the order in which the line of code corresponding to a particular component appears in the *tikzpicture* environment of LaTeX. In the previous example (*pgf4.tex)*, the red circle with a radius of 4cm is drawn first, then the green circle with a radius of 3cm, followed by the blue circle with a radius of 2cm and finally the orange circle with a radius of 1cm. For a better understanding, let us modify the script *pgf4.tex* in such a way that the circles are drawn in the reverse order — the smallest circle is drawn first and so on — to obtain *pgf5.tex*. Now, what will be the image drawn in the output document *pgf5.pdf* when this modified LaTeX file *pgf5.tex* (shown below) is executed with the command *pdflatex pgf5.tex*?

```
\documentclass{article}
\usepackage{tikz}
\begin{document}
\begin{tikzpicture}
     \draw[fill=orange] (0,0) circle (1cm);
     \draw[fill=blue]     (0,0) circle (2cm);
     \draw[fill=green]   (0,0) circle (3cm);
     \draw[fill=red]       (0,0) circle (4cm);
\end{tikzpicture}
\end{document}
```

You might be a bit surprised to see that a single red circle with a radius of 4cm alone is drawn in the output file *pgf5.pdf*. I assure you, we didn't do anything wrong. It's just the way PGF/TikZ works. Try to find out the reason for this abnormal behaviour. If you are unable to find an answer, I have added it as a comment in the file *pgf5.tex* available for download.

Loops in PGF/TikZ

So far, we have drawn two rectangles, three triangles and four circles. What happens if we are asked to draw a hundred triangles or a thousand circles? Well, that's why we have loops in PGF/TikZ to help us. PGF/TikZ has many efficient looping and control structures to help us draw a large number of images with relative ease. Consider the example script *pgf6.tex* shown below, which uses the *foreach* loop provided by PGF/TikZ.

```
\documentclass{article}
\usepackage{tikz}
\begin{document}
\begin{tikzpicture}
     \foreach \x in {1,2,...,9} {
        \foreach \y in {1,2,3} {
          \draw[fill=red!\x\y!green] (\x,\y) circle (4mm);


        }
     }
\end{tikzpicture}
\end{document}
```

The line of code *\foreach \x in {1,2,...,9} {* defines an outer loop that iterates nine times. The line of code *\foreach \y in {1,2,3} {* defines an inner loop that iterates three times. Thus the line of code *\draw[fill=red!\x0!green] (\x,\y) circle*